# Evaluating Machine Learning-Based Intrusion Detection Models Under Distribution Shift

**Author:** Dhansika.R
**Affiliation:** Independent Researcher
**Date:** November 2026
**Correspondence:** dhansika.r.cs@gmail.com

## Abstract

Machine learning (ML)-based network intrusion detection systems (NIDS) are widely deployed in cloud environments. The standard evaluation methodology trains a model on a labelled dataset, splits it randomly, and reports metrics assuming that training and deployment data share a distribution, an assumption that frequently fails in real networks. This study evaluates four models (Logistic Regression, Random Forest, XGBoost, and Multi-Layer Perceptron) under deployment-relevant distribution shifts, compared with conventional random-split evaluation. Using the Canadian Institute for Cybersecurity Intrusion Detection System 2017 (CICIDS2017) dataset, the evaluation covers a conventional random split, a joint session and attack-family shift, attack-family leave-one-out, and synthetic feature perturbation. Two controls support the analysis: a matched-size control that isolates distribution shift from training set size, and a seen-class covariate-shift control that separates feature displacement from unseen-class generalisation.

Per-feature Wasserstein distances quantify shift magnitude, and a threshold sweep tests whether recalibration can recover collapsed F1. Models achieving near-perfect F1 (the harmonic mean of precision and recall; 0.999+) under random splitting can collapse under shift: Random Forest falls from 0.9997 to 0.0000. A matched-size control on same-distribution data yields 0.9992, so the degradation is distributional rather than a data-scarcity effect. Failure modes differ by architecture: tree-based models predict all traffic as benign, while linear and neural models retain partial detection. The area under the receiver operating characteristic curve (ROC-AUC) can remain partially informative (0.819 for Random Forest) while F1 collapses. A threshold sweep recovers much of the lost F1 (Random Forest: 0.0000 to 0.826), but only at below-default thresholds with a substantial false-alarm burden.

**Keywords:** intrusion detection, distribution shift, robustness evaluation, machine learning, cybersecurity, concept drift

## 1. Introduction

Cloud computing has changed how networks are built. A modern deployment runs thousands of interconnected services, creating virtual machines and containers as demand changes, so the traffic flowing through it never stays the same for long. User behaviour shifts, applications are updated, and new threats appear. This makes network security monitoring harder in a concrete way: every new service grows the attack surface, and the traffic mix becomes complex enough that fixed rules cannot keep up with what is benign and what is not.

Network intrusion detection systems (NIDS), also called intrusion detection systems (IDS), are the layer that watches these flows and flags malicious activity: brute-force logins, denial-of-service floods, data exfiltration, and lateral movement. Signature-based tools only catch what they have been taught to look for, so they miss attacks they have never seen. That limitation is why machine learning (ML)-based detection became popular. An ML model learns patterns from labelled traffic and is intended to generalise to new attack variants. In practice, most proposed systems reduce to the same four classifiers: Logistic Regression, Random Forest, XGBoost, and feed-forward neural networks. Reported F1-scores in this literature routinely exceed 0.99 on standard benchmarks.

The question is what that number means. The standard recipe splits one labelled dataset into random training and test halves. Because the split is random, both halves come from the same underlying distribution, with

similar class proportions and similar feature statistics. Under those conditions, near-perfect scores are the expected outcome. The real question is whether they survive deployment, where identical training and test distributions never exist.

## 1.1 The Evaluation–Deployment Gap

There is a gap between how ML-based intrusion detection models are evaluated and how they are deployed. In evaluation, a model is trained and tested on data from the same distribution. In deployment, it faces traffic that differs in temporal patterns, attack composition, and network configuration. The result can be a misleading sense of security: a model that looks high-performing on benchmarks can fail silently in production.

Consider a concrete scenario. An organisation trains a Random Forest classifier on one week of its network traffic. Held-out traffic from that same week gives it an F1-score of 99.97%. Two months later, the network is hit by a distributed denial-of-service (DDoS) attack the model has never seen. Because the model learned decision boundaries specific to its training distribution, it classifies every flow as benign, so detection capability falls to zero while the organisation's dashboards continue to report a healthy system.

This is not a hypothetical edge case; it is a predictable consequence of evaluating a model under conditions that do not match its deployment environment. A paper reporting "F1 = 0.9997 on CICIDS2017" says little about whether that performance survives when the traffic distribution shifts, and in any production network such a shift is inevitable. The problem is worse in cloud environments, where traffic is aggregated from many tenants and services into composite distributions that no single training week can represent. An ML-based NIDS trained on one configuration must generalise to configurations it has never seen. Conventional evaluation does not measure how well it can.

## 1.2 Research Gaps

The ML-based intrusion detection literature contains hundreds of studies reporting high performance on benchmark datasets. Three gaps limit how much practical value those results have.

**First, distribution shift is rarely evaluated.** The dominant practice is a single random train/test split, which does not test robustness to distributional change. Studies that do evaluate under shift often train on one dataset and test on another (cross-dataset evaluation), where differing feature extraction, preprocessing, and network configurations confound the comparison. A confound is a factor that could explain a result instead of the factor under study. What is missing are controlled experiments within a single dataset that vary the distribution while holding everything else constant.

**Second, confounds are rarely isolated.** When a model performs poorly under distribution shift, it is unclear whether the cause is insufficient training data, the distributional difference itself, or their interaction. The conclusion stays ambiguous without experiments that vary these factors independently. A matched-size control, which trains on the same number of samples from the same distribution as the test set, is one such experiment.

**Third, failure modes are rarely analysed.** A model that degrades can fail by predicting everything benign, by predicting everything attack, or by becoming effectively random. These failure modes have different security consequences and need different fixes, but most papers do not look closely at how their model failed.

This paper addresses all three gaps with a controlled experimental design using matched-size controls, multiple shift types, and explicit failure-mode analysis.

## 1.3 Research Question and Contributions

This study addresses one research question (RQ):

**RQ: To what extent does conventional random-split evaluation overestimate the operational effectiveness of machine learning-based intrusion detection models under distribution shift?**

Four sub-questions structure the experiments:

- **RQ1 (Experiment B):** Does distribution shift reflecting realistic deployment conditions cause material performance degradation relative to random-split evaluation?
- **RQ2 (Experiment B2):** Is any such degradation attributable to distributional differences rather than insufficient training data?
- **RQ3 (Experiment B4):** Is the degradation driven primarily by unseen attack classes or by covariate shift?
- **RQ4 (Section 4.8):** To what degree can decision-threshold recalibration recover collapsed performance, and at what false-alarm cost?

The study covers three shift types relevant to network security. In **session and attack-family shift**, training and test data come from sessions with different attack compositions. In **attack-family shift**, an attack type is entirely absent from training. **Synthetic feature perturbation** approximates infrastructure-level changes in observed features. These are different dimensions of the same problem: temporal or session variation, coverage of the attack space, and feature-level change.

If evaluation does not account for distribution shift, benchmark numbers will keep overstating how well these models perform in the field.

The paper makes four contributions. First, a controlled experimental framework that isolates distribution shift from confounding variables such as training set size. It uses matched-size controls with zero train/test overlap and a seen-class covariate-shift control that separates unseen attack classes from feature displacement. Second, a quantitative characterisation of the shifts using per-feature Wasserstein distances. The results show that near-perfect benchmark performance can mask severe vulnerability to distribution shift, with tree-based models dropping from F1 = 0.9997 to F1 = 0.0000. Third, a demonstration that the four evaluated architectures fail differently: tree-based models fail by predicting all traffic as benign, while linear and neural models retain partial detection. Fourth, a threshold-sweep analysis showing that the retained ROC-AUC signal can be converted back into classification performance only at far-below-default decision thresholds. This recovery carries a measurable false-alarm cost.

# 2. Literature Review

## 2.1 Machine Learning for Network Intrusion Detection

ML-based intrusion detection has been studied for over two decades. The KDD Cup 99 dataset was analysed by Tavallaee, Bagheri, Lu, and Ghorbani, who identified its inherent biases and motivated the NSL-KDD benchmark [1]. Sharafaldin, Habibi Lashkari, and Ghorbani later developed CICIDS2017, which includes modern attack types and realistic traffic patterns generated with the CICFlowMeter tool [2]. These datasets became standard benchmarks, and hundreds of studies have evaluated classifiers on them.

The common approaches are Random Forest, gradient-boosted methods such as XGBoost, deep architectures (multi-layer perceptrons (MLPs), convolutional neural networks (CNNs), and autoencoders), and traditional methods such as Logistic Regression and Support Vector Machines. Gradient boosting fits trees sequentially, with each tree correcting errors made by the ensemble so far. A CNN learns local patterns through convolution filters, an autoencoder is trained to reconstruct its input and can flag anomalies by reconstruction error, and a Support Vector Machine separates classes with a maximum-margin boundary. Published performance on these benchmarks frequently exceeds 0.99 F1 under conventional evaluation. The reported scores are high, but the methodology (a single random train/test split) limits what they can reveal about real deployment, where traffic changes over time.

## 2.2 Distribution Shift in Machine Learning

Distribution shift is the phenomenon where the statistical properties of data change between training and deployment, and it is well studied in general machine learning. It includes covariate shift (the input distribution changes while the probability of the label given the input, written $P(Y|X)$, stays the same), prior probability shift (class proportions change), and concept shift (the relationship between inputs and labels changes). All three occur in network security: configurations change (covariate), attack prevalence varies (prior), and attackers evolve their techniques (concept).

Shyaa, Ibrahim, Zainol, Abdullah, Anbar, and Alzubaidi surveyed 71 studies of machine learning and deep learning for intrusion detection, with an emphasis on concept and feature drift, classifying drift as sudden, gradual, incremental, and recurrent, and identifying domain-specific causes such as protocol mutation, attacker strategy evolution, and infrastructure changes [3]. The survey discusses general-purpose drift detectors such as ADWIN (Adaptive Windowing), DDM (Drift Detection Method), and the Page-Hinkley test, and notes that their effectiveness varies by drift type and network context, which suggests that no single drift detection strategy fits all IDS deployments. A related concern is shortcut learning, in which a model exploits superficial regularities that hold in its training data but not under deployment conditions; Geirhos, Jacobsen, Michaelis, Zemel, Brendel, Bethge, and Wichmann document this failure mode across domains [4].

## 2.3 Temporal Evaluation of NIDS

Intrusion detection is inherently a time-series problem: models trained on past traffic must classify future traffic, whose statistical properties evolve. Ahmad, Shahid Khan, Wai Shiang, Abdullah, and Ahmad systematically reviewed machine learning and deep learning approaches for NIDS and observed that many of the reviewed evaluations rely on static benchmark data rather than temporally ordered or streaming data, so reported scores say little about performance as traffic evolves [5]. Parlanti and Catania reached a similar conclusion in a review of more than 40 temporal IDS studies published between 2020 and 2025: static per-flow analysis remains common, and widely used datasets emphasise late-stage attacks, both of which limit what temporal evaluation can reveal [6]. This study complements that literature with controlled experiments that quantify degradation under session-based shift and isolate its cause through matched-size controls.

## 2.4 Adversarial Robustness of NIDS

A separate line of work concerns adversarial robustness, the deliberate modification of traffic to evade detection. This study examines natural distribution shift (session differences and attack-family variation) rather than adversarial manipulation, but the broader robustness framing is relevant. Espindola, Santin, Casimiro, Ferreira, and Viegas surveyed 94 studies published between 2022 and 2025 and mapped the literature across threat models, attack methods, datasets, and metrics [7]. Threat models are formal descriptions of an attacker's goals and capabilities. The survey found a persistent bias toward feature-space manipulation and evasion-centric evaluation, where evasion means crafting inputs that a model misclassifies, and it identifies poisoning (corrupting training data) and model extraction (inferring a model's behaviour through queries) as further threat categories. He, Kim, and Asghar reviewed adversarial attacks and defences for NIDS and reached a similar conclusion: threat models and evaluation practices vary widely, which makes robustness claims difficult to compare [8]. Both surveys treat adversarial manipulation as distinct from natural distribution shift, but the underlying concern is the same, namely that performance measured under one set of conditions may not transfer to another.

## 2.5 Concept Drift Adaptation

Sliding windows, ensembles, and online learning are common drift adaptation methods. An ensemble combines predictions from several simpler models, and online learning updates a model as new data arrive. Shyaa et al. note that retraining can be triggered when drift is detected, periodically, or by incrementally adding knowledge, and that every strategy depends on labelled data [3]. Yang, Guo, Hao, Ciptadi, Ahmadzadeh, Xing, and Wang proposed CADE, which detects concept drift at the level of individual samples and explains why each sample is drifting, rather than treating drift as a single global event; this sample-level view implies that a single global retraining schedule may not fit all attack types [9]. Olímpio, Camargos, Miani, and Faria analysed how delayed labelling and active learning strategies affect model updates in intrusion detection, finding that active learning (selectively requesting labels for the most informative samples) can keep performance close to baseline with only a small set of labelled instances, which reduces but does not remove the operational burden of analyst feedback [10]. These approaches focus on detecting and responding to shift; this study instead quantifies the degradation that occurs when shift is not handled at all.

## 2.6 Summary

This paper contributes a controlled experimental framework that isolates the effects of distribution shift, compares the robustness of several model architectures under identical conditions, and connects quantitative degradation patterns to practical security implications. Unlike prior work that evaluates a single shift type or fails to control for training set size, this study uses matched-size controls with zero train/test overlap to establish what causes the observed degradation. On the basis of this literature, the expected outcome is that models whose decision boundaries depend on individual feature thresholds will degrade most under session and attack-family shift, because they cannot extrapolate to feature values outside their training range, while linear and neural models will degrade more gradually. The experiments test this expectation directly.

# 3. Method

## 3.1 Dataset

This study uses the Friday subset of CICIDS2017, a widely used public benchmark for intrusion detection research [2]. The raw file contains 703,245 flow records. Preprocessing first replaces infinite values with NaN (Not a Number) and drops rows containing NaN or infinity: 527 rows (0.07% of the raw file) were removed, and no imputation was applied. The remaining 702,718 records contain 413,933 benign flows and 288,785 attack flows.

Non-numeric columns are then dropped, leaving 78 numeric features. To keep the experiments computationally manageable, the benign class was randomly downsampled to 29,452 flows with random seed 42, while the Bot, DDoS, and PortScan classes were retained in full. This step is a deliberate subsample, not a property of the original dataset. It produces 318,237 flow records across four classes:

| Class label | Flow count | Share of dataset (%) |
|---|---|---|
| BENIGN | 29,452 | 9.3 |
| Bot | 1,956 | 0.6 |
| DDoS | 128,025 | 40.2 |
| PortScan | 158,804 | 49.9 |

***Table 1:*** *Class distribution in the Friday subset of CICIDS2017 after preprocessing (318,237 flow records across four classes).*

Because the benign class was reduced, the processed dataset is attack-dominated: DDoS and PortScan together make up 90.1% of the data, while Bot is 0.6% and benign is 9.3%. The raw Friday file is benign-dominated (58.9% benign), so the processed imbalance is a consequence of the downsampling and the choice of day, not a claim about production traffic. This matters for interpretation: accuracy and false-positive rates computed on the processed test sets are not directly transferable to a benign-dominated network. The imbalance also complicates training, so the experimental design reports precision, recall, F1, and the false positive rate (FPR) alongside accuracy.

Features are flow-level statistics (Flow Duration, Total Fwd Packets, Total Backward Packets, packet-size statistics), flag-based features (counts of the Transmission Control Protocol (TCP) flags: synchronise (SYN), acknowledge (ACK), reset (RST), and push (PSH)), and inter-arrival time (IAT) statistics. All are numeric and represent aggregated network flow characteristics. Flow duration and inter-arrival times are recorded in microseconds; packet lengths and byte counts are in bytes; packet and flag counts are dimensionless counts.

## 3.2 Prediction Task

The primary evaluation task is binary classification: BENIGN (0) versus ATTACK (1). The four-class problem is evaluated as a supplementary analysis in Section 4.10.

## 3.3 Models

Four models are evaluated, each representing a different algorithmic paradigm:

- **Logistic Regression (LR):** Linear classifier with L2 (ridge) regularisation (a penalty on the squared magnitude of the model weights); it requires feature scaling because the penalty and the gradient-based fit are sensitive to feature magnitude, and it assumes a linear decision boundary.
- **Random Forest (RF):** Ensemble of 80 decision trees, each grown to a maximum depth of 15 levels. Each split tests one feature against a threshold (an axis-aligned split). A trained forest keeps those thresholds fixed, so rescaling a feature at test time can move flows across a threshold learned on the original scale.
- **XGBoost (XGB):** Gradient-boosted ensemble of 80 trees, each limited to a depth of 5 levels, with a learning rate of 0.1. Gradient boosting fits trees sequentially so that each tree corrects errors made by the ensemble so far.
- **Multi-Layer Perceptron (MLP):** Feed-forward neural network with two hidden layers of 128 and 64 units; it captures nonlinear feature interactions but needs sufficient training data.

All models use fixed hyperparameters (settings chosen before training) and a single random seed of 42 (the starting value for the pseudo-random number generator), so a rerun with the same software versions reproduces the same results. The experiments were implemented in Python, with pandas and NumPy for data handling, scikit-learn for Logistic Regression, Random Forest, the MLP, and preprocessing, XGBoost for gradient boosting, and Matplotlib for the figures.

**Focus on traditional machine learning.** The study is deliberately confined to these four models rather than deep architectures. The motivating deployment context is a throughput-constrained cloud IDS. Random Forest, gradient-boosted trees, and logistic regression remain common in production because they run at near-line-rate inference (applying a trained model to new data) on commodity central processing units (CPUs) with modest memory and power needs. Deep models typically need graphics processing unit (GPU) acceleration and accept higher latency. The inductive bias of a model, meaning the assumptions it makes about the form of the pattern, also matters for the observed failures. Random Forest's axis-aligned splits cannot extrapolate beyond the range of feature values seen in training, consistent with its complete collapse to predicting all traffic as benign when test features fall outside the training range. Linear and neural models learn continuous, magnitude-sensitive representations that degrade more gradually. Comparing deep architectures against these models is outside the scope of this study and left as future work.

## 3.4 Evaluation Scenarios

**Experiment A: Conventional Random Split.** A stratified split, which preserves class proportions in both parts, divides the data so that 80% is used for training and 20% for testing. This provides the conventional benchmark metric and serves as the baseline.

**Experiment B: Joint Session and Attack-Family Distribution Shift.** Training data comprise all 1,956 Bot flows plus a disjoint random 40% of the 29,452 benign flows, giving 13,736 training flows (85.8% benign). Test data comprise all 128,025 DDoS and 158,804 PortScan flows plus a different random 40% of the benign flows, giving 298,609 test flows (3.9% benign). The remaining 20% of benign flows are unused. This construction approximates the Friday morning and afternoon session division, because Bot attacks occurred in the morning session and DDoS and PortScan attacks in the afternoon session, but it is not a timestamp-level split: the CSV concatenates flows by class, and the original ordering is not preserved. It is therefore described as a joint session and attack-family shift rather than a purely temporal shift. Critically, B is also an **open-set** condition: the test set contains two attack families (DDoS and PortScan) entirely absent from training, so it conflates covariate shift (session timing, benign composition, feature statistics), prior-probability shift (training is 85.8% benign; test is 3.9% benign), and unseen-class generalisation (novel attack signatures). Experiment B4 (Section 3.5) isolates the covariate-shift component.

**Experiment C: Attack-Family Leave-One-Out.** For each attack type (Bot, DDoS, PortScan), the model trains on all other traffic and tests on the held-out attack type plus as many unseen benign samples as the benign pool allows, capped so that at least 1,000 benign flows remain in training. To prevent leakage, meaning information from the test set influencing training, benign samples used in the test set are explicitly excluded from training. This directly evaluates how well models generalise to entirely unseen attack types, a critical capability for deployment where the training set inevitably misses some attack variants.

**Experiment D: Synthetic Feature Perturbation.** Using the same random split as Experiment A, the test set features are perturbed synthetically: Flow Duration is multiplied by random factors drawn uniformly from 2.0

to 5.0, and the packet count features (Total Fwd Packets and Total Backward Packets) are scaled by Gaussian noise (normally distributed random variation) with a mean of 1.5 and a standard deviation of 0.5, then clipped to the range from minus one million to one million. This tests invariance to feature-level changes from infrastructure modification, sensor recalibration, or changes in flow measurement tools.

## 3.5 Control Experiments

The key methodological contribution is the matched-size control, which isolates distribution shift from training set size.

**Experiment B2: Matched-Size Control.** The training pool from Experiment A (80% of the processed data) is randomly subsampled to 13,736 flows, matching B's training-set size, and evaluated on Experiment A's held-out 20% test split. The subsample uses a different random seed (123) from the Experiment A split. This control has: (1) the same training-set size as B, (2) the same test set as Experiment A, (3) zero train/test overlap, and (4) training data from the same distribution as the test set. Comparing B with B2 isolates the effect of distribution shift while controlling for training-set size. One difference remains: the B2 subsample is attack-dominated (90.8% attack), whereas B's training set is 85.8% benign. Section 5.4 treats this class-composition difference as a limitation.

**Experiment B4: Seen-Class Covariate-Shift Control.** Experiment B conflates covariate shift with unseen-class generalisation; B4 isolates the covariate-shift part. Only DDoS and BENIGN traffic are used (157,477 flows: 128,025 DDoS and 29,452 benign), divided by a stratified 80/20 split into 125,981 training flows and 31,496 test flows. All models are trained on the training portion, so **DDoS is present in the training set**, and evaluated on the held-out portion. In B4a (no-shift control) the held-out test is used unmodified. In B4b (covariate shift) the same perturbation design as Experiment D is applied to every held-out flow, including the benign flows. Because the attack class is present in training, any degradation between B4a and B4b is attributable to feature displacement (covariate shift), in isolation from unseen-class generalisation. The contrast between B4b and B quantifies how much of B's collapse is associated with features outside the training range versus entirely novel attack families.

## 3.6 Evaluation Metrics

Accuracy, precision, recall, F1-score, the false positive rate (FPR), and the area under the receiver operating characteristic curve (ROC-AUC) are reported for binary classification. For multiclass results, weighted averages are reported, where each class receives a weight proportional to its number of flows.

**Accuracy:** the proportion of all flows that are classified correctly.

**Precision:** the proportion of predicted attacks that are actually attacks.

**Recall:** the proportion of actual attacks that the model detects. Recall is also called the detection rate.

**F1-score:** the harmonic mean of precision and recall, a single metric that balances false positives and false negatives.

**FPR:** the proportion of benign traffic incorrectly classified as attack. In intrusion detection, this is the false alarm rate.

**ROC-AUC:** the area under the receiver operating characteristic curve, which plots the true positive rate against the false positive rate at every decision threshold. It can be read as the probability that a randomly chosen attack flow is ranked above a randomly chosen benign flow, where 0.5 corresponds to chance and 1.0 to perfect ranking.

## 3.7 Quantifying Shift Magnitude

The one-dimensional Wasserstein distance (W1; also called the Earth Mover's Distance) is used to measure how far the compared feature distributions drifted. It measures the minimum amount of work needed to transform one distribution into the other. For each feature f, W1(f) = W(P_train(f), P_test(f)) is computed on a random subsample of 25,000 points per side and normalised by the pooled standard deviation of the two distributions, a combined measure of their spread, so distances are dimensionless and comparable across features. Of the 78 features, 68 have a non-zero pooled standard deviation; the other 10 are constant in both samples and are skipped. The mean normalised Wasserstein distance across those 68 features is reported as a single shift-magnitude index, together with the top shifted features. This is computed for the B shift (B training versus B test features), the D shift (unmodified versus perturbed test features), and the B4 shift (unmodified versus perturbed B4 test features), and related to the observed F1 drop.

## 3.8 Threshold Sweep

To test whether the retained ROC-AUC signal can be turned back into usable classification, the decision threshold is swept on the Experiment B test set from 0.0 to 1.0 in steps of 0.001 for each model. F1, precision, recall, and FPR are computed at every threshold. F1 is reported at the default threshold (0.50) versus the best non-degenerate operating point, defined as the maximum F1 over thresholds from 0.001 up to but not including 0.50. Threshold zero is excluded because it predicts every flow as attack and therefore produces a degenerate all-attack classifier. Because the B test set is 96.1% attack, FPR and the fraction of samples flagged as attack are also reported at each operating point, so the false-alarm cost of aggressive thresholds is explicit.

## 3.9 Experimental Procedure

All experiments follow the same procedure per model:

1. Fit a standard scaler (scikit-learn's StandardScaler) on the training data only, then transform the test data with the same fitted scaler
2. Train the model on the scaled training data
3. Predict labels and probabilities on the scaled test data
4. Compute all evaluation metrics

Each experiment trains and evaluates all four models independently. A fixed seed is set for every library used; the main split uses 42, and the B2 subsample uses 123. The shift types cover three deployment-relevant dimensions: session or temporal variation, attack-space coverage, and feature-level infrastructure change.

# 4. Results

The raw data are the labelled flow records in the Friday subset of CICIDS2017 described in Section 3.1. Every number in the tables below is a derived metric computed from the model predictions on those records, using the definitions in Section 3.6. Because all experiments share the same preprocessed data and the same fixed seed, differences between tables reflect the experimental condition rather than changes in the underlying dataset.

## 4.1 Conventional Random Split (Experiment A)

Under conventional random splitting, all models achieve near-perfect performance (Table 2). Random Forest and XGBoost reach F1 = 0.9997, Logistic Regression reaches 0.9951, and MLP reaches 0.9989. These results match the wider literature on CICIDS2017, where models routinely exceed 0.99 F1 under standard evaluation. Precision ranges from 0.9938 to 0.9997 and recall from 0.9965 to 0.9998, and false positive rates are low across the board (0.0029 to 0.0611). Inference times differ by about an order of magnitude (LR below 0.0001 ms per sample versus RF at 0.0014 ms per sample; ms denotes milliseconds); end-to-end deployment latency would also depend on preprocessing, hardware, and batching. Figures 1 and 2 place this landscape next to the other evaluation scenarios.

| Model | Accuracy | F1-score | Precision | Recall | FPR | Inference time (ms per sample) |
|---|---|---|---|---|---|---|
| LR | 0.9911 | 0.9951 | 0.9938 | 0.9965 | 0.0611 | <0.0001 |
| RF | 0.9995 | 0.9997 | 0.9997 | 0.9998 | 0.0029 | 0.0014 |
| XGB | 0.9995 | 0.9997 | 0.9996 | 0.9998 | 0.0037 | 0.0004 |
| MLP | 0.9981 | 0.9989 | 0.9985 | 0.9994 | 0.0148 | 0.0009 |

***Table 2:*** *Experiment A results. Near-perfect performance across all models under conventional random splitting.*




***Figure 1:*** *Model performance (accuracy, F1, recall) across the random-split, session-shift, matched-size, and no-shift evaluation scenarios.*

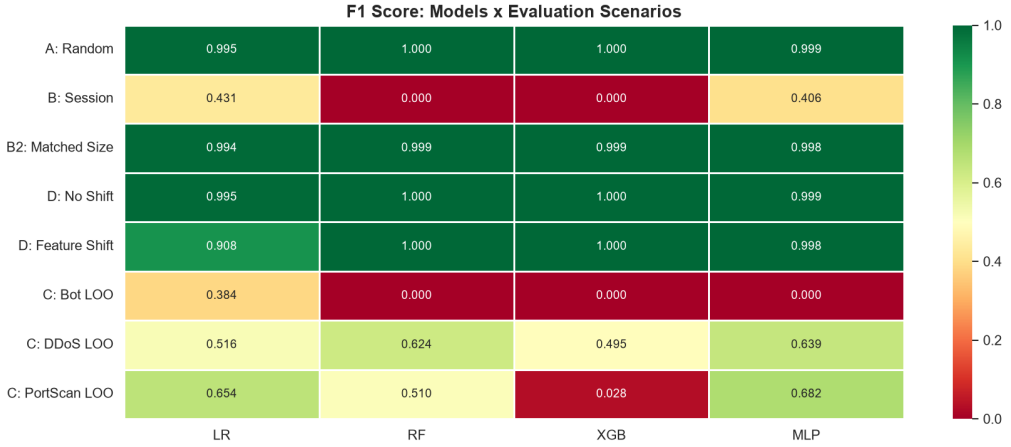



***Figure 2:*** *F1-score landscape across models and evaluation scenarios, including the attack-family leave-one-out results.*

## 4.2 Joint Session and Attack-Family Shift (Experiment B)

Under this shift, performance degrades substantially (Table 3). Random Forest collapses to F1 = 0.0000, predicting all traffic as benign. XGBoost similarly collapses to F1 = 0.0003. Logistic Regression retains partial detection (F1 = 0.4307) and MLP limited detection (F1 = 0.4059), but both miss roughly 73% of attacks. Since B combines unseen attack classes (DDoS and PortScan) with session-level covariate shift and a class-composition shift, Experiment B4 and the Wasserstein quantification in Section 4.7 attribute this degradation to its constituent causes.

| Model | Accuracy | F1-score | Precision | Recall | FPR | Inference time (ms per sample) |
|---|---|---|---|---|---|---|
| LR | 0.3021 | 0.4307 | 0.9949 | 0.2748 | 0.0343 | 0.0001 |
| RF | 0.0393 | 0.0000 | 0.0000 | 0.0000 | 0.0040 | 0.0014 |
| XGB | 0.0395 | 0.0003 | 0.6282 | 0.0002 | 0.0025 | 0.0003 |
| MLP | 0.2833 | 0.4059 | 0.9960 | 0.2549 | 0.0249 | 0.0007 |

***Table 3:*** *Experiment B results. Severe degradation, especially for tree-based models.*

The low accuracy for RF (0.0393) and XGB (0.0395) is counter-intuitive at first, given that the test set is 96.1% attack. It is explained by the models predicting benign for almost everything: a model that predicts all-benign on a test set that is only 3.9% benign gets roughly 3.9% accuracy. The low FPR values (0.0040 and 0.0025) are not positive signals either. They are consequences of total detection failure, because the models predict attack for almost no flows.

## 4.3 Isolating Distribution Shift from Training Set Size (B versus B2)

Comparing B against its matched-size control B2 (Table 4) isolates the role of distribution shift. Both use 13,736 training samples. The main difference is whether training data come from the same distribution as the test set (B2) or a different one (B).

| Model | B: Session Shift | B2: Same Distribution |
|---|---|---|
| LR | 0.4307 | 0.9942 |
| RF | 0.0000 | 0.9992 |
| XGB | 0.0003 | 0.9991 |
| MLP | 0.4059 | 0.9979 |

***Table 4:*** *Distribution shift isolation. B and B2 use the same training set size (13,736) with zero train/test overlap.*

The contrast is substantial. With the same 13,736 training samples, RF achieves F1 = 0.9992 under same-distribution conditions (B2) but F1 = 0.0000 under distribution shift (B). This is strong evidence that the degradation is distributional, not a shortage of training data. One difference remains: the B2 subsample is 90.8% attack, whereas B's training set is 85.8% benign, so B2 also differs in class composition. That difference is part of the distribution shift under study, but it means the B–B2 gap combines covariate, prior-probability, and unseen-class effects rather than isolating covariate shift alone.

The implication is direct: 13,736 training samples are sufficient when they match the test distribution. They produce near-perfect performance under those conditions. The problem is that the training data, though from the same dataset, come from sessions and attack types that do not represent the test distribution. That is a fundamentally different problem from data scarcity and needs different solutions (distribution-aware evaluation, continual retraining, or domain adaptation, which adjusts a model to a new input distribution) rather than collecting more labels. Figure 3 shows this contrast directly.

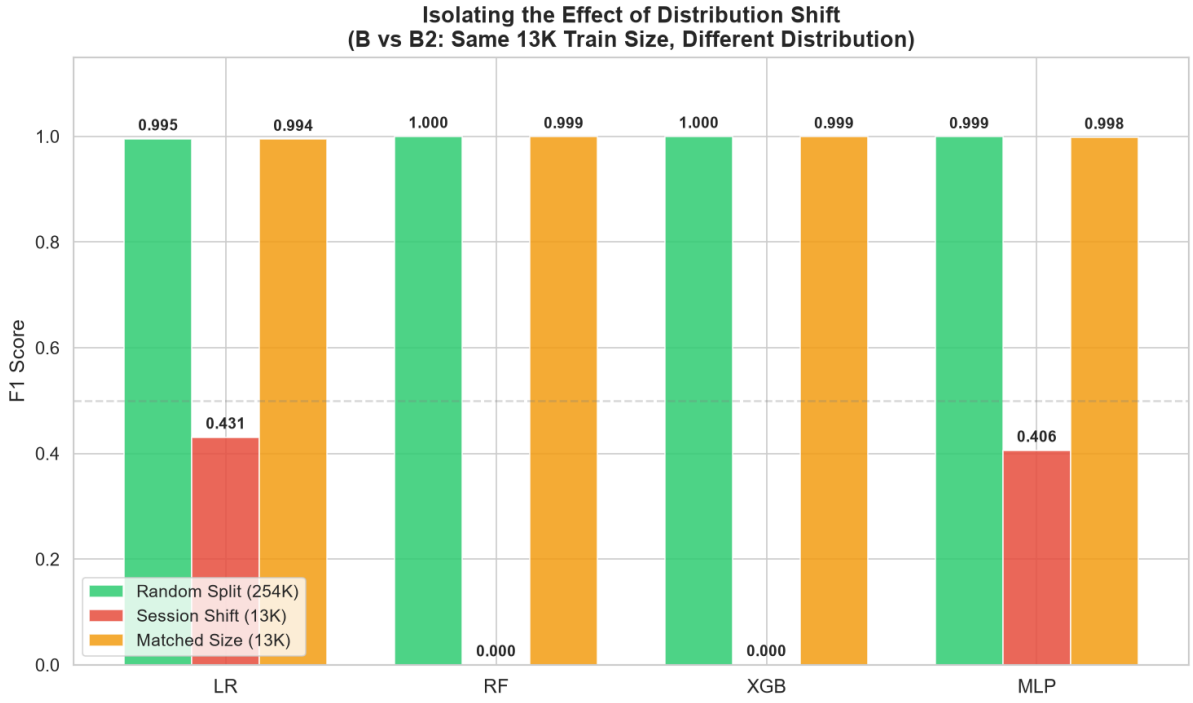



***Figure 3:*** *Isolating distribution shift from training set size. B and B2 use identically sized training sets (13,736 samples) whose distributions differ.*

## 4.4 Separating Unseen-Class Shift from Covariate Shift (Experiment B4)

Experiment B's test set contains attack families absent from training, so its collapse could in principle be blamed on either novel attack signatures or displaced feature values. Experiment B4 removes the unseen-class component: DDoS is present in training, and the held-out feature values are perturbed for every test flow (Table 5).

| Model | B4a: Same Distribution | B4b: Covariate Shift | Change in F1 |
|---|---|---|---|
| LR | 0.9990 | 0.9982 | -0.0007 |
| RF | 0.9999 | 0.9997 | -0.0002 |
| XGB | 0.9999 | 0.9997 | -0.0003 |
| MLP | 0.9998 | 0.9995 | -0.0004 |

***Table 5:*** *Seen-class covariate-shift control. DDoS is present in both training and test; every test flow, including benign flows, has its features perturbed. F1 drops are negligible compared with Experiment B.*

When the attack class is present in training, feature displacement of the same design as Experiment D causes negligible degradation (F1 drops of 0.0002–0.0007 across all four models). This contrasts strongly with Experiment B, where the same models lost 0.56–0.99 F1. The dominant driver of B's collapse is therefore the presence of entirely novel attack families (unseen-class generalisation) rather than feature displacement per se. The covariate-shift part of B cannot be fully separated from its class-composition change here, but B4 shows that displacement alone, even a moderate one (normalised Wasserstein 0.152, Section 4.7), does not produce B's severe failures. This frames B as an open-set recognition problem, where open-set recognition means classification when the test set contains classes absent from training: the models have no decision boundary for attack families they have never seen.

## 4.5 Attack-Family Leave-One-Out (Experiment C)

When specific attack types are excluded from training (Table 6), the results reveal which families are most distinct from the training distribution.

| Held-out attack family | LR F1 | RF F1 | XGB F1 | MLP F1 |
|---|---|---|---|---|
| Bot | 0.384 | 0.000 | 0.000 | 0.000 |
| DDoS | 0.516 | 0.624 | 0.495 | 0.639 |
| PortScan | 0.654 | 0.510 | 0.028 | 0.682 |

***Table 6:*** *F1 scores under attack-family leave-one-out. Bot detection is most severely affected.*

The Bot leave-one-out is the most severe: all models except LR reach F1 = 0.000 when Bot is absent from training, despite 314,325 training samples from the other three classes. Bot traffic is evidently highly distinct from DDoS, PortScan, and benign traffic in feature space, and learned patterns do not transfer. DDoS and PortScan leave-one-out degrade more gracefully, with MLP best (0.639 and 0.682). The more graceful DDoS and PortScan results suggest that those families share more feature structure with the remaining training classes than Bot does. "Unseen attack type" is thus not a uniform category: some unseen attacks are more detectable, depending on their feature similarity to known attacks. Figure 4 plots the leave-one-out F1 scores; Figure 5 converts them into relative degradation from the baseline.

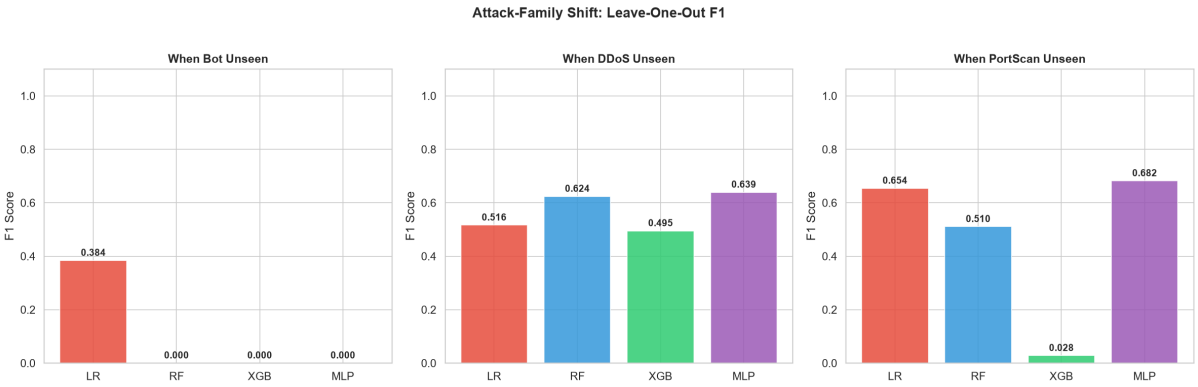



***Figure 4:*** *Attack-family leave-one-out F1 per held-out class. Bot detection is the most severely affected.*

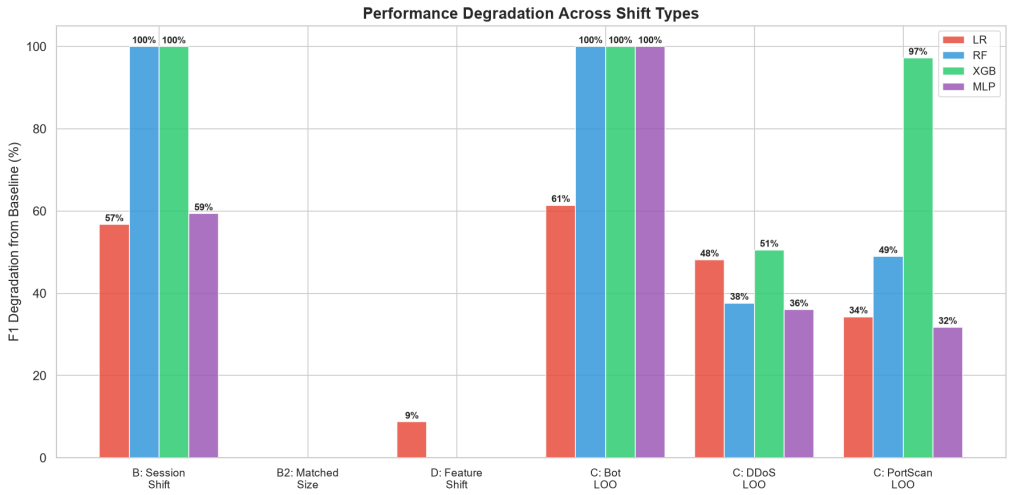



***Figure 5:*** *F1 degradation relative to the Experiment A baseline for each shift condition and model.*

## 4.6 Synthetic Feature Perturbation (Experiment D)

Under synthetic feature perturbation (Table 7), tree-based models are largely unaffected (RF and XGB both stay at F1 = 0.9997), while Logistic Regression shows clear degradation (0.9951 to 0.9084).

| Model | F1 without shift | F1 under feature shift |
|---|---|---|
| LR | 0.9951 | 0.9084 |
| RF | 0.9997 | 0.9997 |
| XGB | 0.9997 | 0.9997 |
| MLP | 0.9989 | 0.9983 |

***Table 7:*** *Experiment D results. Tree models showed little degradation under the specific feature perturbations used in this experiment.*

RF and XGBoost barely moved under this perturbation design (F1 effectively unchanged at 0.9997). This is an empirical result for this particular design, not evidence of general invariance to feature distribution changes. The tree-based result is a property of the particular trees learned here, not a general invariance guarantee. A decision tree partitions on individual features at fixed thresholds. Multiplying a feature by a positive constant preserves the ordering of samples on that feature, and a tree could represent the same partition by rescaling its thresholds; however, a tree that has already been trained keeps its original thresholds, so shifted test values can cross them. Why the perturbation had almost no effect therefore depends on where the learned splits sit relative to the shifted values. In these forests and boosted ensembles, the affected features appear not to have been decisive near the operating region, or the shifts did not move enough test flows across the learned cut points. Linear models, by contrast, combine features additively with learned weights, so multiplying feature magnitudes changes the linear score directly and moves samples relative to the 0.5 threshold; Logistic Regression therefore degrades as observed (0.9951 to 0.9084).

The contrast is informative. Tree models were highly vulnerable to session and attack-family shift (Experiment B) but stable under this particular perturbation. More realistic perturbations, such as changed feature correlations, new features from protocol changes, or dropped features from sensor failures, could affect tree models differently.

## 4.7 Shift Magnitude Quantification (Wasserstein Distances)

The preceding experiments describe shifts qualitatively; Table 8 quantifies them. For each shift, the mean normalised one-dimensional Wasserstein distance between the two feature distributions compared in that shift is reported (a random subsample of 25,000 points per side, per-feature distance divided by the pooled standard deviation, across the 68 features with non-zero pooled standard deviation).

| Shift | Mean normalised W1 | Top shifted features | Maximum F1 drop |
|---|---|---|---|
| B (joint session + unseen class) | 0.293 | PSH Flag Count (1.01), Min Packet Length (0.81), Bwd Packet Length Min (0.77) | 0.9997 (RF) |
| B4 (seen-class covariate shift) | 0.152 | Packet Length Variance (0.94), Flow IAT Std (0.71), Flow Duration (0.71) | 0.0007 (LR) |
| D (synthetic feature perturbation) | 0.106 | Packet Length Variance (0.59), Flow Duration (0.47), Flow IAT Std (0.47) | 0.0868 (LR) |

***Table 8:*** *Mean normalised Wasserstein distance between the compared feature distributions in each shift, with the largest per-feature distances and the largest observed F1 drop across the four models (baseline = the experiment's own no-shift condition: random split A for B, B4a for B4, no-shift control for D).*

Three observations follow. First, B is the largest displacement (0.293) and produces the largest F1 collapse, in the expected direction. Second, the relationship between displacement and degradation is strongly non-linear: B4 displaces features roughly half as much as B (0.152 versus 0.293) yet produces about a thousand-fold smaller F1 drop. Third, B's F1 drop of 0.9997 is far out of proportion to what feature displacement alone would predict from B4, indicating that unseen-class generalisation, not covariate magnitude, is the primary driver of B's collapse. Figure 6 reports this relationship directly.

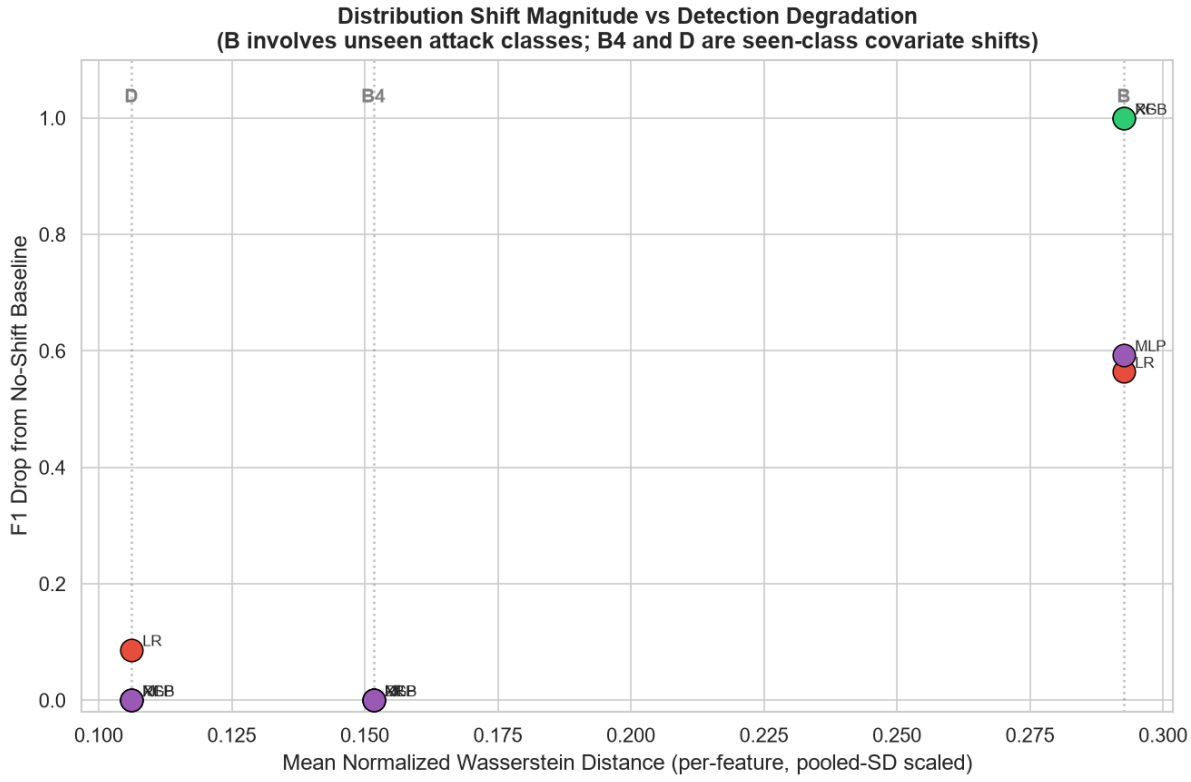



***Figure 6:*** *Mean normalised Wasserstein shift magnitude versus maximum F1 drop across the B, B4, and D shifts.*

## 4.8 Threshold Sweep on the Shifted Test Set

The ROC-AUC analysis below shows that rankings survive even when F1 collapses at the default threshold; the sweep (Section 3.8) tests whether this signal is usable. Table 9 reports F1 and FPR at the default threshold (0.50), at a moderately aggressive threshold (0.05), and at an aggressive threshold (0.01); Figure 7 shows the full curves.

| Model | F1 @ 0.50 | FPR @ 0.50 | F1 @ 0.05 | FPR @ 0.05 | F1 @ 0.01 | FPR @ 0.01 |
|---|---|---|---|---|---|---|
| LR | 0.4307 | 0.0343 | 0.7104 | 0.0581 | 0.7115 | 0.1602 |
| RF | 0.0000 | 0.0040 | 0.7114 | 0.0399 | 0.8262 | 0.0654 |
| XGB | 0.0003 | 0.0025 | 0.4635 | 0.0081 | 0.6037 | 0.0219 |
| MLP | 0.4059 | 0.0249 | 0.4923 | 0.0368 | 0.6614 | 0.0492 |

***Table 9:*** *Threshold sweep on the Experiment B test set. Lowering the decision threshold recovers a substantial fraction of F1 at the cost of higher FPR; even at the best non-degenerate thresholds, all models still underperform their same-distribution baselines.*

Over the full fine-grained sweep from 0.0 to 1.0, the best non-degenerate operating points are LR 0.7240 at 0.001, RF 0.8263 at 0.009, XGB 0.6731 at 0.002, and MLP 0.7067 at 0.001. At those operating points the false-positive rates are 0.300 (LR), 0.065 (RF), 0.054 (XGB), and 0.076 (MLP), and the models flag between 49.1% (XGB) and 68.1% (RF) of all test flows as attack. Threshold-based recalibration therefore recovers part of the collapsed F1: Random Forest rises from 0.0000 (FPR 0.4%) at the default threshold to 0.7114 at 0.05 and 0.8262 near 0.01, with FPR climbing to 4.0% and 6.5%. The ranking signal identified by ROC-AUC is therefore real and exploitable, but recovery is neither free nor complete. First, the best non-degenerate thresholds (0.001–0.009) are far below the default, so near-line-rate deployments would flag a large share of traffic; the FPR cost looks diluted on this 96.1%-attack test set but would be far larger in benign-dominated production traffic. Second, if threshold zero is allowed, the naive best-F1 point is the degenerate solution of predicting every sample as attack (F1 = 0.98 at FPR = 1.0), which provides no useful detection; this is why the analysis uses the non-degenerate definition. Third, even the best recoverable operating point (RF, 0.8263) falls below same-distribution capability (B2, 0.9992): recalibration is a partial remedy, not a restoration of the original decision function.

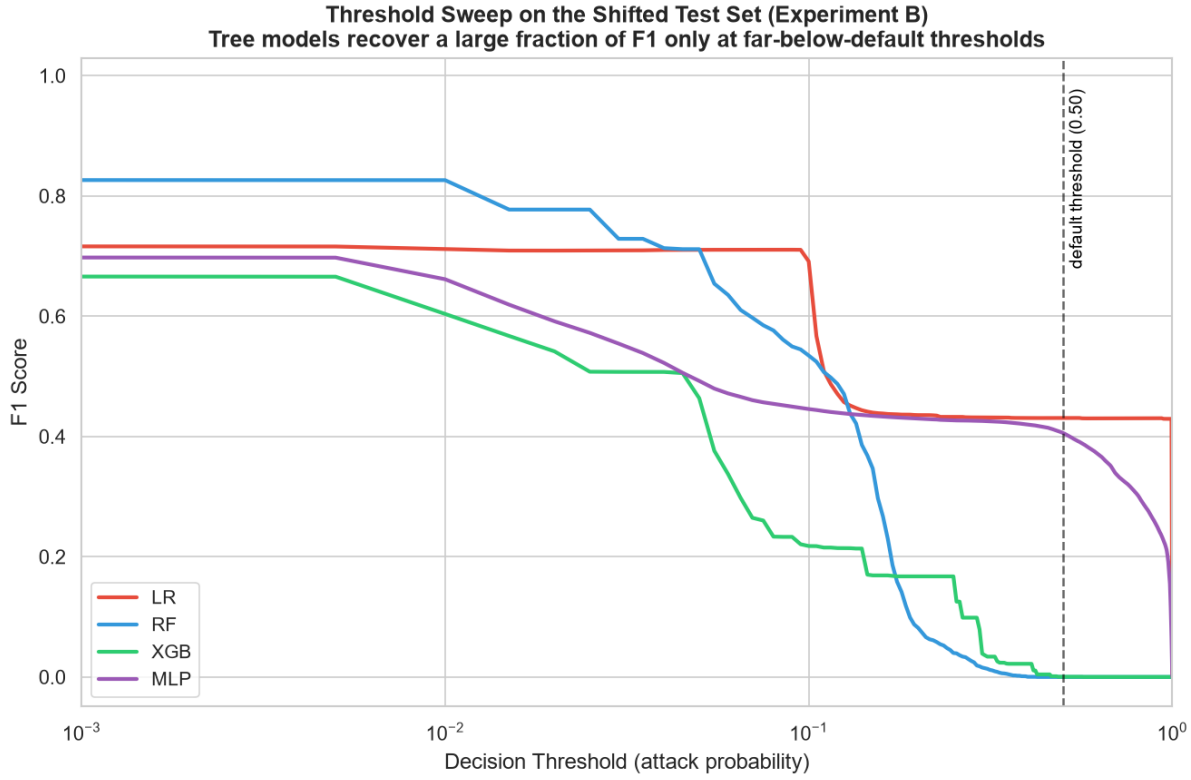



***Figure 7:*** *F1 versus decision threshold on the Experiment B test set. Tree models require far-below-default thresholds to recover useful F1.*

## 4.9 ROC-AUC Under Shift

An important secondary finding emerges from comparing F1 and ROC-AUC under session-based shift (Table 10). Figure 8 compares ROC-AUC across scenarios.

| Model | F1 (B) | ROC-AUC (B) |
|---|---|---|
| LR | 0.4307 | 0.6081 |
| RF | 0.0000 | 0.8185 |
| XGB | 0.0003 | 0.8452 |
| MLP | 0.4059 | 0.6485 |

***Table 10:*** *F1 collapses while ROC-AUC retains partial signal under distribution shift.*

RF achieves ROC-AUC = 0.8185 despite F1 = 0.0000, and XGB achieves 0.8452 despite F1 = 0.0003. This suggests the models' probability rankings retain some discriminative information, while the default classification threshold is poorly suited to the shifted test distribution. For RF and XGB, the pattern is consistent with a calibration failure, where calibration means the agreement between predicted probabilities and observed outcomes: the ranking survives, but the default operating point does not. For LR and MLP, ranking quality itself is weak (ROC-AUC 0.6081 and 0.6485), so recalibration has much less to recover. The threshold sweep in Section 4.8 supports this interpretation: the retained ranking signal can be converted back into classification performance, but only at far-below-default thresholds with a measurable false-alarm cost, so the recoverable signal is bounded. Monitoring ROC-AUC alongside F1 in production could provide an early-warning signal, one that distinguishes threshold miscalibration from fundamental model failure. The sweep shows, however, that the practical remedy is limited.

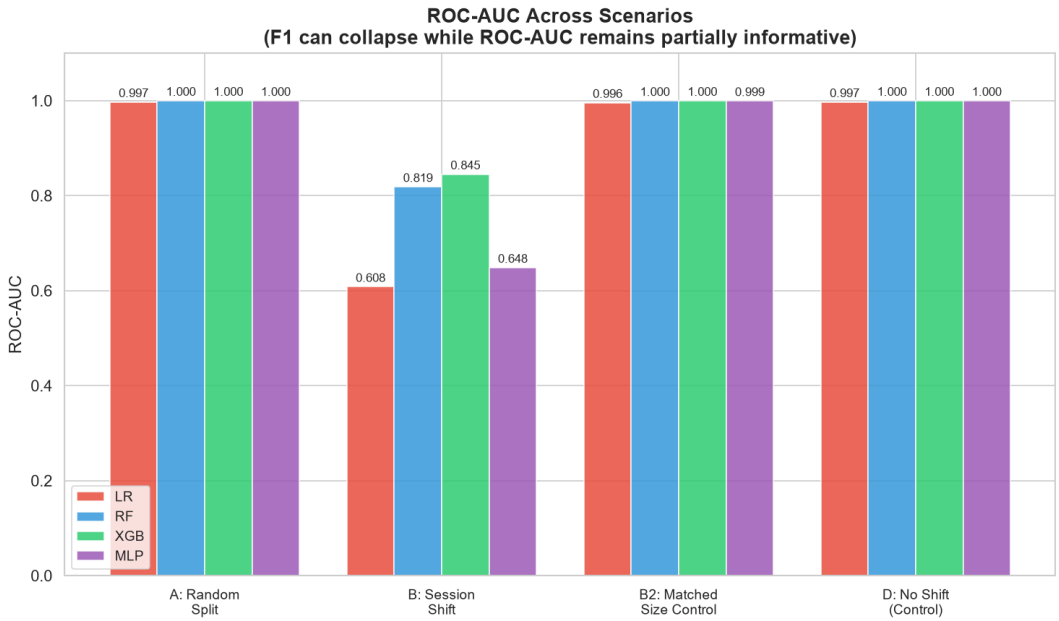



***Figure 8:*** *ROC-AUC across evaluation scenarios. F1 can collapse while ROC-AUC retains a partial ranking signal.*

## 4.10 Multi-Class Results

As a supplementary check, the same four models were trained on the four-class problem (BENIGN, Bot, DDoS, PortScan) using the Experiment A random split. Weighted-average F1-scores were 0.9956 (LR), 0.9995 (RF), 0.9996 (XGB), and 0.9986 (MLP). The ordering matches the binary results, and the high scores are consistent with the same-distribution caveat: the random split does not test robustness to shift. These results are reported for completeness; the binary experiments remain the primary evidence because they isolate the shift conditions.

# 5. Discussion and Analysis

The expected pattern set out in Section 2.6 was that models whose decision boundaries depend on individual feature thresholds would degrade most under session and attack-family shift, while linear and neural models would degrade more gradually. The results match this expectation closely. Random Forest and XGBoost, the two tree-based models, collapsed to near-zero F1 in Experiment B, whereas Logistic Regression and the MLP retained partial detection. Because the random-split results also reproduce the high scores commonly reported for CICIDS2017, with all four models above 0.99 F1, the later collapse is unlikely to be an implementation fault. The conventional benchmark behaves as published; the difference appears only when the evaluation condition changes.

## 5.1 The False Sense of Security Problem

The central security implication is that conventional evaluation can produce a false sense of security. A model reported as 99.97% F1 may provide zero detection capability in deployment, and this is a predictable consequence of the gap between evaluation and deployment conditions.

The pattern is counter-intuitive: the models with the highest benchmark scores in this experiment (RF and XGB at 0.9997) also failed the most severely under the evaluated shift. Predicting all traffic as benign means they provide effectively no security value while appearing, by conventional metrics, to be the strongest performers. This supports the emphasis of the National Institute of Standards and Technology (NIST) Artificial Intelligence (AI) Risk Management Framework on continuous monitoring and adaptation [11]. It also aligns with the NIST Cyber AI Profile, which treats AI-specific risk management as an ongoing activity [12]. A model that performs well at one point may fail in a changed environment, and standard evaluation will not detect this before deployment.

## 5.2 Failure Mode Analysis

The failure modes differ by architecture, with different security consequences:

**Tree-based models (RF and XGB):** Fail by predicting all traffic as benign (recall approaches 0). This is the most dangerous failure mode: effectively every attack passes undetected while the model creates the appearance of a clean network.

**Linear and neural models (LR and MLP):** Fail by detecting only a subset of attacks (recall approximately 0.25–0.27) with high precision. They miss roughly 73% of attacks but rarely raise false alarms. This is imperfect, but the failure mode keeps at least partial visibility into threats.

From an operational standpoint, the tree-based failure mode is the more dangerous of the two: a model that almost never raises an alarm provides no security value, and deploying it may even increase risk by replacing signature-based systems that would have caught some attacks. The practical implication for model selection in cloud security is that the model that performs best under conventional evaluation may not be the most robust choice for deployment. Organisations should evaluate candidates under distribution-aware conditions before deploying rather than trusting benchmark scores alone. If distribution-aware evaluation is not feasible, these results suggest that, on this dataset, LR or MLP may be a safer choice than their lower benchmark scores imply.

## 5.3 Implications for Cloud Deployment

Deploying ML-based intrusion detection in clouds adds complexity. In centralised architectures where traffic from multiple services is fed into a single model, the distribution shift problem is amplified. The finding that tree-based models collapse under the joint session and attack-family shift used here suggests that any model expected to generalise across diverse traffic types is especially at risk.

**Inference latency.** Inference times differ by an order of magnitude (LR below 0.0001 ms/sample, MLP 0.0007, RF 0.0014). At cloud scale, where millions of flow records may be classified per minute, these differences are operationally significant. But the latency advantage of LR is meaningless if the model fails to detect attacks under shift, so the speed-robustness trade-off should be informed by distribution-aware evaluation.

**Threshold sensitivity.** The threshold sweep shows that the retained ranking signal is exploitable but bounded. All four models kept ROC-AUC above 0.5 under shift (0.61 to 0.85), so an operator could recover part of F1 by lowering the threshold. The limitation is that the best non-degenerate thresholds lie between 0.001 and 0.009, far from 0.50, at which point FPR and the share of flagged traffic rise materially. Random Forest's FPR, for example, increases from 0.4% at threshold 0.50 to 6.5% at threshold 0.01, and LR's rises from 3.4% to 30.0% at its best threshold of 0.001. A real deployment in benign-dominated traffic would see a proportionally larger false-alarm volume than the 96.1%-attack test set suggests. Picking the operating threshold without ground-truth labels from the target distribution is a practical challenge.

**Hybrid architectures.** The differing failure modes across architectures suggest that ensembles mixing model types may be more robust. Tree-based models offer fast inference and high benchmark performance; linear models degrade more gradually under shift. A system combining both could in principle take the strengths of each while covering their weaknesses.

## 5.4 Limitations

**Dataset scope.** Only the Friday subset of CICIDS2017 is used (BENIGN, Bot, DDoS, and PortScan). The full dataset includes additional attack categories (Brute Force, Heartbleed, Web Attacks, and Infiltration) spread across Monday through Friday, which may produce different shift patterns, and these findings may not generalise to datasets with different attack types or network configurations.

**Session-shift construction.** In Experiment B, the split is built by attack family (Bot in training; DDoS and PortScan in test) and by disjoint benign subsamples, which approximates the morning and afternoon session division but is not a timestamp-level split. Preprocessing concatenates rows by class, partially destroying the

original temporal ordering. It is therefore called session and attack-family shift rather than temporal shift.

**Train-test size imbalance.** The session split produces substantially different training and test set sizes (13,736 versus 298,609). The B2 control indicates that distribution shift, not training set size alone, is the primary cause of degradation. The imbalance also shifts class proportions (training is 85.8% benign; test is 3.9% benign), an additional variable in the design. B2 also differs from B in class composition: its random subsample is attack-dominated (90.8% attack), whereas B's training set is benign-dominated. B2 therefore controls training-set size and distribution match but not class balance, and part of the B–B2 gap may reflect prior-probability shift as well as covariate and unseen-class shift.

**Model configurations.** Four architectures are evaluated with fixed, untuned hyperparameters; other configurations might have different robustness profiles. However, the complete collapse of RF and XGB under session shift suggests the main finding is robust to moderate hyperparameter variation. Recurrent networks (which carry information across time steps), attention-based transformers (which weigh input elements by learned relevance), and graph neural networks (which operate on graph-structured data) are not evaluated, so they may behave differently; the ML focus here is deliberate (Section 3.3).

**Feature shift design.** The synthetic perturbation in Experiment D is limited: it multiplicatively rescales three flow-level features with independent random factors, leaving the remaining features and the overall joint structure largely unchanged. Tree models showed little degradation under this transformation, but that does not establish robustness to joint-distribution, correlation, feature-availability, or sensor changes, any of which could reorder samples across tree splits. Joint-distribution shifts remain untested.

**Statistical uncertainty.** Every experiment uses a single fixed seed, and no confidence intervals or significance tests are reported. Repeated runs with different seeds would be needed to attach error bars to the metrics. The main collapse is qualitative and far larger than seed-to-seed variation is likely to be, so the central conclusion is unlikely to depend on this choice. The smaller differences, however, such as LR versus MLP in Experiment B or the 0.0002–0.0007 changes in B4, should be read as indicative rather than precise.

**Binary classification focus.** While multi-class results are reported in Section 4.10, the primary analysis uses binary classification (BENIGN versus ATTACK). Distribution shift may affect multi-class performance differently, especially class-specific detection.

## 5.5 Future Work

Several directions follow from these limitations. First, the framework should be replicated on additional datasets, including UNSW-NB15, CICIDS2018, and real production traffic, and on architectures not evaluated here, such as recurrent networks, transformers, and calibrated variants including Platt-scaled or isotonic-regression classifiers. Platt scaling fits a logistic model to classifier scores, and isotonic regression is a non-parametric monotonic calibration method. Second, drift detection methods that alert operators when operating conditions diverge from training conditions would complement robustness evaluation, and the ROC-AUC-versus-F1 gap identified here is a candidate early-warning signal. Third, domain adaptation and continual learning, the latter updating a model incrementally as new data arrive, could address shift during deployment rather than only measuring it beforehand. Fourth, adaptive operating-point selection that accounts for false-alarm cost in benign-dominated traffic could make the threshold-sweep finding usable in practice. Finally, repeating every experiment across several random seeds would allow confidence intervals to be reported alongside the point estimates.

# 6. Conclusion

This study asked whether conventional random-split evaluation overestimates the operational effectiveness of ML-based intrusion detection models under distribution shift. The evidence indicates that it does, and the margin is large. Two models scoring 0.9997 F1 on a random split collapsed to 0.0000 and 0.0003 under distribution shift, while a matched-size control trained on the same number of samples from the same distribution held at 0.999. The collapse is therefore distributional rather than a shortage of training data.

The failure modes differ by architecture in ways that matter for security. Tree-based models, which earn the highest benchmark scores, fail most severely by predicting all traffic as benign, while linear and neural models retain partial detection. The seen-class covariate-shift control attributes the collapse in the joint session and attack-family scenario primarily to unseen attack classes rather than to feature displacement. ROC-AUC remains partially informative while F1 collapses, so the models retain ranking information despite broken operational behaviour; a threshold sweep can recover part of that signal, but only at far-below-default thresholds whose false-alarm cost must be borne in benign-dominated traffic.

These findings have implications for both research methodology and operational deployment. For researchers, random-split evaluation remains a useful upper bound on capability, but it should not be the only reported result, and any study claiming high performance on an intrusion detection benchmark should add at least one distribution-shift evaluation. For practitioners, benchmark performance alone is an unsafe basis for model selection: candidates should be evaluated under conditions that resemble deployment, drift should be monitored in production, and fallback detection mechanisms should remain independent of ML assumptions. The broader lesson is that evaluation methodology shapes the conclusions drawn about model capability, because a model evaluated under conditions that do not match deployment is effectively a different model.

## Funding Statement and Acknowledgements

**Funding:** This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

**Acknowledgements:** The author thanks the maintainers of the open-source tools used in this study (Python, NumPy, pandas, scikit-learn, XGBoost, and Matplotlib) for making reproducible computational research possible. Dhansika.R is the sole author and was responsible for conceptualisation, research direction, dataset selection, experimental design, results review and interpretation, and writing.

**Use of AI-assisted technologies:** Generative AI tools were used as guided assistance. The author conceived the research direction, selected the dataset, and made all high-level experimental design decisions, including the matched-size control (B2) and the covariate-shift control (B4). AI assistance was used to draft the experiment scripts, generate the figures, and edit the manuscript prose. The author directed the experiments, reviewed every result, verified the reported metrics against the experiment logs, and takes full responsibility for the scientific validity and integrity of the work.

**Data and code availability:** All code (preprocessing, experiments, verification, and figure generation) and the derived result files are available from the project repository. The CICIDS2017 dataset is publicly available from the Canadian Institute for Cybersecurity at the University of New Brunswick; it is not redistributed in the repository. The preprocessing pipeline is fully scripted and reproducible, and `code/verify_numbers.py` checks every metric reported in the results tables against the saved experiment outputs.

**Repository:** https://github.com/dhansikarcs-cs/ids-distribution-shift

**Competing interests:** The author declares no competing interests.

## References

[1] Tavallaee, Mahbod, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. "A Detailed Analysis of the KDD CUP 99 Data Set." In *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 1–6. 2009.

[2] Sharafaldin, Iman, Arash Habibi Lashkari, and Ali A. Ghorbani. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization." In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 108–116. 2018.

[3] Shyaa, Methaq A., Noor Farizah Ibrahim, Zurinahni Zainol, Rosni Abdullah, Mohammed Anbar, and Laith Alzubaidi. "Evolving Cybersecurity Frontiers: A Comprehensive Survey on Concept Drift and Feature

Dynamics Aware Machine and Deep Learning in Intrusion Detection Systems." *Engineering Applications of Artificial Intelligence* 137, Part A (2024): 109143.

[4] Geirhos, Robert, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. "Shortcut Learning in Deep Neural Networks." *Nature Machine Intelligence* 2, no. 11 (2020): 665–673.

[5] Ahmad, Z., A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad. "Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches." *Transactions on Emerging Telecommunications Technologies* 32, no. 1 (2021): e4150.

[6] Parlanti, Tatiana S., and Carlos A. Catania. "Temporal Analysis Framework for Intrusion Detection Systems: A Novel Taxonomy for Time-Aware Cybersecurity." *arXiv* 2511.03799 (2025).

[7] Espindola, Allan da S., Altair O. Santin, António Casimiro, Pedro M. Ferreira, and Eduardo K. Viegas. "Understanding the Adversary: A Survey of Adversarial Machine Learning in Network Intrusion Detection." *Computer Science Review* 62 (2026): 100995.

[8] He, Ke, Dan Dongseong Kim, and Muhammad Rizwan Asghar. "Adversarial Machine Learning for Network Intrusion Detection Systems: A Comprehensive Survey." *IEEE Communications Surveys and Tutorials* 25, no. 1 (2023): 538–566.

[9] Yang, Limin, Wenbo Guo, Qingying Hao, Arridhana Ciptadi, Ali Ahmadzadeh, Xinyu Xing, and Gang Wang. "CADE: Detecting and Explaining Concept Drift Samples for Security Applications." In *Proceedings of the 30th USENIX Security Symposium*, 2327–2344. 2021.

[10] Olímpio, Gilberto, Jr., Lasaro Camargos, Rodrigo Sanches Miani, and Elaine Ribeiro Faria. "Model Update for Intrusion Detection: Analyzing the Performance of Delayed Labeling and Active Learning Strategies." *Computers and Security* 134 (2023): 103451.

[11] National Institute of Standards and Technology. *AI Risk Management Framework (AI RMF 1.0)*. Gaithersburg, MD: NIST, 2023.

[12] National Institute of Standards and Technology. *Cybersecurity Framework Profile for Artificial Intelligence (Cyber AI Profile): NIST Community Profile*. NIST IR 8596 (Initial Preliminary Draft). Gaithersburg, MD: NIST, 2025.

## Biography

**Name:** Dhansika.R

**Age:** 15

**Affiliation:** Independent Researcher

**Photograph:** [author photograph to be supplied]

Dhansika.R is an independent researcher with an interest in cybersecurity, cloud computing, and applied artificial intelligence. Her research examines how distribution shift affects machine learning-based intrusion detection, and she maintains the public code and verification scripts for her study. She is also interested in psychology and in making technical work reproducible and honest.